

割裂元年

老师在讲 Spring Boot 2.7。

前端是 Vue 2。

他说，这学期要做一个图书管理系统。要求不算复杂：图书增删改查，书架增删改查，再接一个大模型问答。

比如你问：

“这本书在哪里？”

大模型要告诉你。

我听到这里的时候，觉得还行。

图书表里有书架编号，书架表里有位置。用户问一句，模型识别意图，后端查数据库，再把结果说成人话。再简单一点，让模型生成只读 SQL，后端做清洗和权限限制，查到什么答什么。数据库里没有位置，就说“暂无位置信息”。

事情大概就这么大。

后来老师说：

“调用一下大模型 API，这个功能做出来，八十分到九十分应该有了。”

我愣了一下。

我问自己，那数据库呢？

大模型没看过数据库。

它不知道这本书昨天在三号书架，今天被挪到了七号书架。

它甚至不知道学校有没有这本书。

但没关系。

只要它说话了。

八十分到九十分。

我忽然觉得数据库有些多余。

既然如此，不如删了。

反正书在哪里，可以问大模型。

老师又想教我们用 AI 写代码。

他说现在有些同学用 **Codex**，几分钟就把作业做完了。

“这个工具很好用，”他说，“它也算是一个 AI 平台，跟我们网页版复制粘贴其实差不多。”

我低头看了一眼电脑。

没说话。

过了一会儿，他开始下载 VS Code 里的插件。

GitHub 转了半天。

没下来。

又试了一次。

还是没下来。

教室里几十个人等着。

老师看了一会儿屏幕，说：

“可能今天运气不太好。”

“下次再试试。”

我突然觉得这节课一下深奥了起来。

原来计算机科学发展到今天，已经重新引入了天象。

以前网络不通，我们查 DNS、路由、代理、TLS。

现在先进了。

叫：

运气不好。

教室里是两个班，大概六七十个人。

老师说，前面还有两个班。

其他专业还有。

加起来一两百人。

真正用 Codex 的没几个。

有人用。

大概也是：

“帮我做个图书管理系统。”

过几分钟：

“已完成。”

哪里不对，再说：

“帮我修一下。”

再过几分钟：

“已修复。”

至于到底修了什么，为什么坏，数据库有没有写进去，前端看到的東西和数据库是不是同一个世界——

不重要。

AI 会修。

老师也很高兴。

学生也很高兴。

模型最喜欢这种局面。

所有人都完成了任务。

我的版本里，在 `ask()` 最前面还有一道拦截。

你问：

“今天天气怎么样？”

它不会调用 `DeepSeek`。

你问股票，也不会。

你问电影，也不会。

它只会很客气地告诉你：

“抱歉，我无法直接回答这个问题。”

因为这是图书管理系统。

不是慈善机构。

更不是让我替全班同学给大模型公司交 API 费的地方。

如果你问：

“有没有适合计算机专业的书？”

它才进去。

先查馆藏。

有就有。

没有就没有。

不知道就说不知道。

我那时候甚至还没上 RAG，也没用向量数据库。

后来想想，好像也没必要。

书在哪儿这种事情，MySQL 比大模型清楚。

这在以前似乎叫数据库原理。

不知道为什么，到了 AI 时代，又要重新发明一次。

以后大概还会出一本教材。

书名我都替他们想好了：

《人工智能赋能智慧图书馆应用原理》

四百八十页。

第一章，人工智能概述。

第二章，大语言模型发展历史。

第三章，智慧图书馆的时代背景。

第四章，AI 赋能传统数据库。

写到第三百页的时候，大概终于会提到：

查询图书位置时，可考虑访问数据库。

老师还在讲。

我把电脑合上了。

插件依然没有下载下来。

大模型依然值八十分到九十分。

GitHub 今天运气不太好。

我觉得这节课该学的已经学完了。

于是我逃回了宿舍。

在宿舍床上，我反复回味这件事，感觉已经力竭了。

今天，2026 年 9 月 14 日，大概是割裂元年吧。